

Finite Element Method Applied to 1-Dimensional Partial Differential Equation using Python

We seek to determine a finite element approximation to the following partial differential equation:

$$-\frac{d}{dx} \left(-4 \frac{du}{dx} \right) + 8 \frac{du}{dx} + 5u = \sin x + 8 \cos x, \quad -\pi < x < \pi$$

Subject to one Nuemann boundary condition:

$$-4 \frac{du}{dx} = -20, \quad x = -\pi$$

And one Dirichlet boundary condition:

$$u = 2, \quad x = \pi$$

With the true solution:

$$u(x) = 2e^{-x} \left(e^{\pi} \sin \frac{x}{2} + \frac{2(3 - e^{2\pi})}{e^{\pi}} \cos \frac{x}{2} \right) + \sin x$$

First, we seek to find the weak formulation of the given PDE. To do so, we start by multiplying both sides by the test function, v , and integrating:

$$\begin{aligned} \int_{-\pi}^{\pi} \left(-\frac{d}{dx} \left(-4 \frac{du}{dx} \right) + 8 \frac{du}{dx} + 5u \right) v \, dx &= \int_{-\pi}^{\pi} (\sin x + 8 \cos x) v \, dx \\ \int_{-\pi}^{\pi} \left(-\frac{d}{dx} \left(-4 \frac{du}{dx} \right) + 8 \frac{du}{dx} + 5u \right) v \, dx &= \int_{-\pi}^{\pi} \left(-\frac{d}{dx} \left(-4 \frac{du}{dx} \right) \right) v \, dx + 8 \int_{-\pi}^{\pi} \frac{du}{dx} v \, dx + 5 \int_{-\pi}^{\pi} u v \, dx \end{aligned}$$

Focusing on the first term for the left-hand side:

$$\int_{-\pi}^{\pi} \left(-\frac{d}{dx} \left(-4 \frac{du}{dx} \right) \right) v \, dx$$

By using integration by parts:

$$\int_{-\pi}^{\pi} \left(-\frac{d}{dx} \left(-4 \frac{du}{dx} \right) \right) v \, dx = \left[4 \frac{du}{dx} v \right]_{-\pi}^{\pi} - \int_{-\pi}^{\pi} 4 \frac{du}{dx} \frac{dv}{dx} \, dx$$

Applying boundary conditions:

$$-4 \frac{du}{dx} = -20, x = -\pi \rightarrow \frac{du}{dx}(-\pi) = 5$$

Inserting boundary conditions:

$$\int_{-\pi}^{\pi} \left(-\frac{d}{dx} \left(-4 \frac{du}{dx} \right) \right) v dx = \left[4 \frac{du}{dx} v \right]_{-\pi}^{\pi} - \int_{-\pi}^{\pi} 4 \frac{du}{dx} \frac{dv}{dx} dx = 4[-5v(-\pi)] - \int_{-\pi}^{\pi} 4 \frac{du}{dx} \frac{dv}{dx} dx$$

Now, for the middle term of the left hand side:

$$\int_{-\pi}^{\pi} 8v \frac{du}{dx} dx$$

Applying integration by parts:

$$\int_{-\pi}^{\pi} 8v \frac{du}{dx} dx = [8uv]_{-\pi}^{\pi} - \int_{-\pi}^{\pi} 8 \frac{dv}{dx} u(x) dx$$

Using the given boundary conditions, we have:

$$\int_{-\pi}^{\pi} 8v \frac{du}{dx} dx = -8u(-\pi)v(-\pi) - \int_{-\pi}^{\pi} 8 \frac{dv}{dx} u(x) dx$$

Finally, for the third term on the left hand side:

$$\int_{-\pi}^{\pi} 5uv dx$$

Remains unchanged, leading to the following weak form solution:

$$\int_{-\pi}^{\pi} \left(-4 \frac{du}{dx} \frac{dv}{dx} + 8 \frac{dv}{dx} u(x) + 5uv \right) dx = 20v(-\pi) + \int_{-\pi}^{\pi} (\sin x + 8 \cos x) v dx$$

Applying the finite element solution to the weak form using the basis function, ϕ :

$$\int_{-\pi}^{\pi} \left(-4 \frac{du}{dx} \frac{d\phi}{dx} + 8 \frac{d\phi}{dx} u(x) + 5u\phi \right) dx = 20\phi(-\pi) + \int_{-\pi}^{\pi} (\sin x + 8 \cos x) \phi dx$$

Where, assuming a Finite Element solution, we insert the following relationship into the weak form:

$$u(x) \approx \sum_{i=1}^n u_i \phi_i(x)$$

Which causes the weak form to become:

$$\sum_{i=1}^n u_i \int_{-\pi}^{\pi} \left(-4 \frac{d\phi_i}{dx} \frac{d\phi_j}{dx} + 8\phi_i \frac{d\phi_j}{dx} + 5\phi_j \phi_i \right) dx = 20\phi(-\pi) + \int_{-\pi}^{\pi} (\sin x + 8 \cos x) \phi dx$$

Mapping as follows:

$$J^e = \frac{d\phi}{dx} = \frac{h^e}{2}$$

The mapping is performed so that the local matrix is uniform across all elements – alleviating the number of integrals required. For the finite element system, $Ku=f$, we define the following components of the system:

$$K_{ij} = \int_{-1}^1 \left(-4 \frac{d\phi_i}{dx} \frac{d\phi_j}{dx} + 8\phi_i \frac{d\phi_j}{dx} \frac{1}{J^e} + 5\phi_j \phi_i \right) dx$$

$$f_i = 20\phi_i(-\pi) + \int_{-1}^1 (\sin x + 8 \cos x) \phi_i J^e dx$$

$$u = \sum_{i=1}^n u_i$$

Now, to define the basis functions for the quadratic approximation, where in compliance with figure 4.1 from Whitely, there are 3 nodes per element for quadratic basis functions.

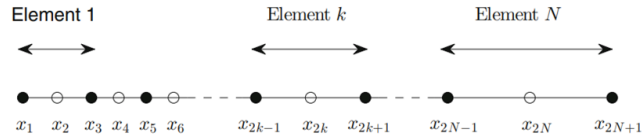


Fig. 4.1 The nodes and elements for a mesh that may be used to calculate a finite element solution that is a quadratic approximation to the true solution on each element. Nodes on the boundary of an element are denoted by a *filled circle*, and nodes in the interior of an element are denoted by an *open circle*

By the Lagrange basis function:

$$\phi_j(x) = \prod_{\substack{i=1 \\ i \neq j}}^n \frac{x - X_i}{X_j - X_i}$$

For quadratic approximation, the value for n is 3.

$$\phi_j(x) = \prod_{\substack{i=1 \\ i \neq j}}^3 \frac{x - X_i}{X_j - X_i}$$

For the first local basis function (on interval 0 and 1):

$$\phi_1(x) = \left(\frac{x - X_2}{X_1 - X_2} \right) \left(\frac{x - X_3}{X_1 - X_3} \right) = \left(\frac{x - \frac{1}{2}}{-\frac{1}{2}} \right) (x - 1) = 2 \left(\frac{1}{2} - x \right) (1 - x)$$

For the second local basis function:

$$\phi_2(x) = \left(\frac{x - X_1}{X_2 - X_1} \right) \left(\frac{x - X_3}{X_2 - X_3} \right) = \left(\frac{x - 0}{\frac{1}{2} - 0} \right) \left(\frac{x - 1}{\frac{1}{2} - 1} \right) = -4x(x - 1) = 4x(1 - x)$$

For the third local basis function:

$$\phi_3(x) = \left(\frac{x - X_1}{X_3 - X_1} \right) \left(\frac{x - X_2}{X_3 - X_2} \right) = \left(\frac{x - 0}{1} \right) \left(\frac{x - \frac{1}{2}}{1 - \frac{1}{2}} \right) = 2x \left(x - \frac{1}{2} \right)$$

Let's say that generally that $x = \frac{(x_b - x_a)}{2}\xi + \frac{(x_b + x_a)}{2}$ or $x = \frac{1}{2}\xi + \frac{1}{2}$:

$$\phi_1(\xi) = 2 \left(\frac{1}{2} - \left(\frac{1}{2}\xi + \frac{1}{2} \right) \right) \left(1 - \left(\frac{1}{2}\xi + \frac{1}{2} \right) \right) = 2 \left(\frac{1}{2}\xi \right) \left(\frac{1}{2} - \frac{1}{2}\xi \right) = \left(\frac{1}{2} \right) \xi(1 - \xi)$$

$$\phi_2(\xi) = 4 \left(\frac{1}{2}\xi + \frac{1}{2} \right) \left(1 - \left(\frac{1}{2}\xi + \frac{1}{2} \right) \right) = 2(\xi + 1) \left(\frac{1}{2} - \frac{1}{2}\xi \right) = (\xi + 1)(1 - \xi) = (1 - \xi^2)$$

$$\phi_3(\xi) = 2 \left(\frac{1}{2}\xi + \frac{1}{2} \right) \left(\frac{1}{2}\xi + \frac{1}{2} - 1 \right) = 2 \left(\frac{1}{2}\xi + \frac{1}{2} \right) \left(\frac{1}{2}\xi \right) = \left(\frac{1}{2} \right) \xi(1 + \xi)$$

We will also explore varying the number of elements per nodes, using the following basis functions. First, for the quadratic approximation:

$$\phi_j(x) = \prod_{\substack{i=1 \\ i \neq j}}^4 \frac{x - X_i}{X_j - X_i}$$

$$\Phi_1(x) = \left(\frac{x - \frac{1}{3}}{0 - \frac{1}{3}} \right) \left(\frac{x - \frac{2}{3}}{0 - \frac{2}{3}} \right) \left(\frac{x - 1}{0 - 1} \right) = \frac{9}{2} \left(\frac{1}{3} - x \right) \left(\frac{2}{3} - x \right) (1 - x)$$

$$\Phi_2(x) = \left(\frac{x - 0}{\frac{1}{3} - 0} \right) \left(\frac{x - \frac{2}{3}}{\frac{1}{3} - \frac{2}{3}} \right) \left(\frac{x - 1}{\frac{1}{3} - 1} \right) = \frac{27}{2} x \left(\frac{2}{3} - x \right) (1 - x)$$

$$\Phi_3(x) = \left(\frac{x - 0}{\frac{2}{3} - 0} \right) \left(\frac{x - \frac{1}{3}}{\frac{2}{3} - \frac{1}{3}} \right) \left(\frac{x - 1}{\frac{2}{3} - 1} \right) = \frac{27}{2} x \left(x - \frac{1}{3} \right) (1 - x)$$

$$\Phi_4(x) = \left(\frac{x-0}{1-0} \right) \left(\frac{x-\frac{1}{3}}{1-\frac{1}{3}} \right) \left(\frac{x-\frac{2}{3}}{1-\frac{2}{3}} \right) = \frac{9}{2}x(x-\frac{1}{3})(x-\frac{2}{3})$$

Now, we will proceed and solve the equation with cubic and quadratic basis functions to determine the difference in accuracy.

Next, the basis functions will be used to define the elements in the local stiffness matrix using the predefined form of the weak form incorporating the Galerkin method. To ensure proper mapping to the conical dimension $[-1,1]$, we use the Jacobian as follows:

$$K_{ij} = \int_{-1}^1 \left(-4 \frac{d\phi_i}{dx} \frac{d\phi_j}{dx} \frac{1}{J^e} + 8\phi_i \frac{d\phi_j}{dx} + 5\phi_j \phi_i J^e \right) dx$$

So, for the first element in the first local matrix, we get:

$$K_{11} = \int_{-1}^1 \left(-4 \frac{d\phi_1}{dx} \frac{d\phi_1}{dx} \frac{1}{J^e} + 8\phi_1 \frac{d\phi_1}{dx} + 5\phi_1 \phi_1 J^e \right) dx$$

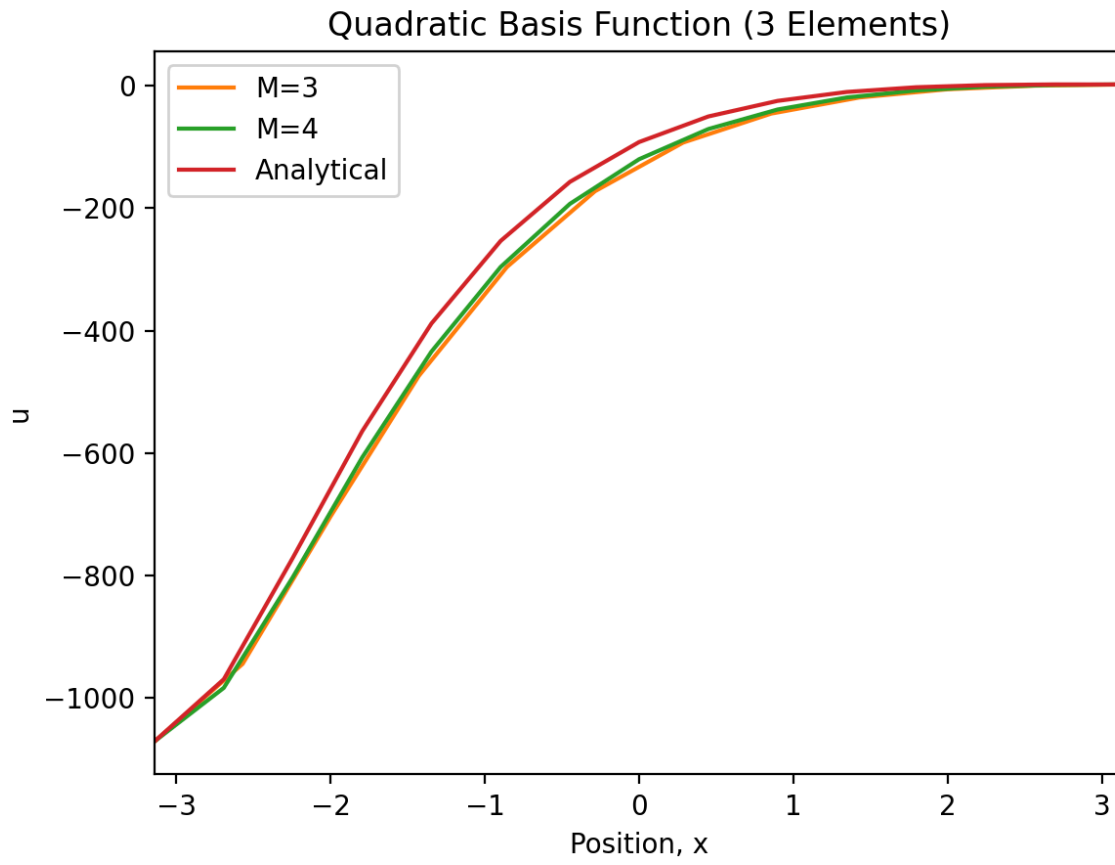
Then, once we assemble the complete stiffness matrix and force vector as follows:

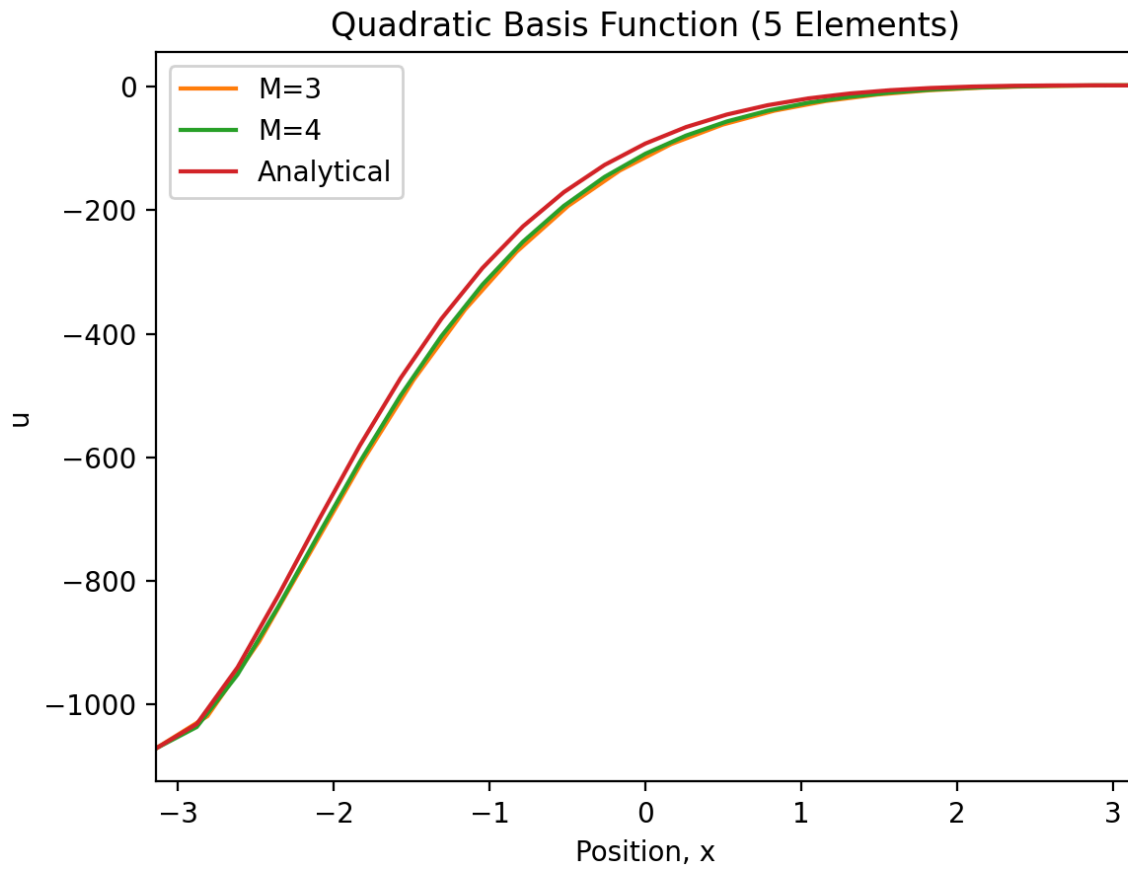
$$\begin{bmatrix} K_{11} & K_{12} & \cdots & K_{1N} \\ K_{21} & K_{22} & \cdots & K_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ K_{N1} & K_{N2} & \cdots & K_{NN} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_N \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_N \end{bmatrix}$$

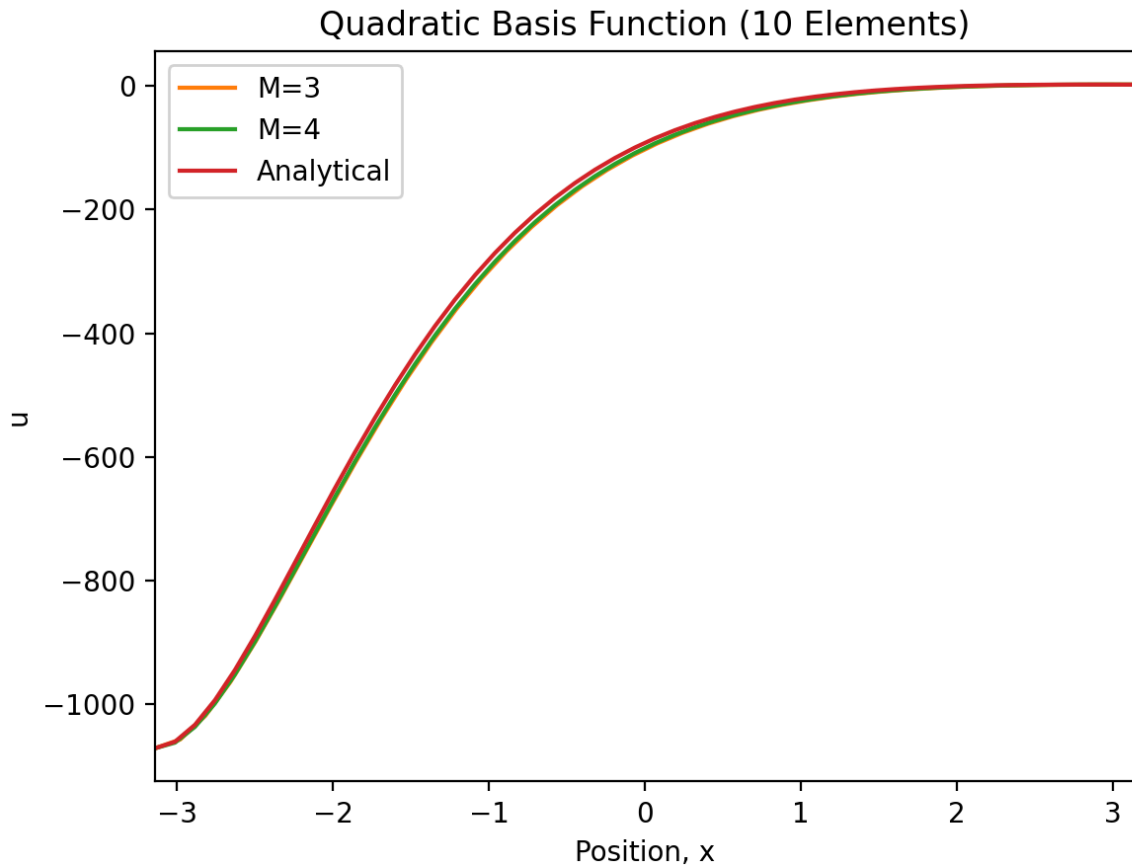
Where, since we know that $u(x=\pi)=2$, then we can reduce the system as follows:

$$\begin{bmatrix} K_{11} & \cdots & K_{1(N-1)} \\ \vdots & \ddots & \vdots \\ K_{(N-1)1} & \cdots & K_{(N-1)(N-1)} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{N-1} \end{bmatrix} = \begin{bmatrix} f_1 - u_N K_{1N} \\ f_2 - u_N K_{2N} \\ \vdots \\ f_{N-1} + u_N K_{NN} \end{bmatrix}$$

To analyze the results, we will plot the analytical solution along with the cubic and quadratic basis function finite element approximations. Then, the L^2 errors will be plotted separately. Starting with the results for the 3 element approximations.

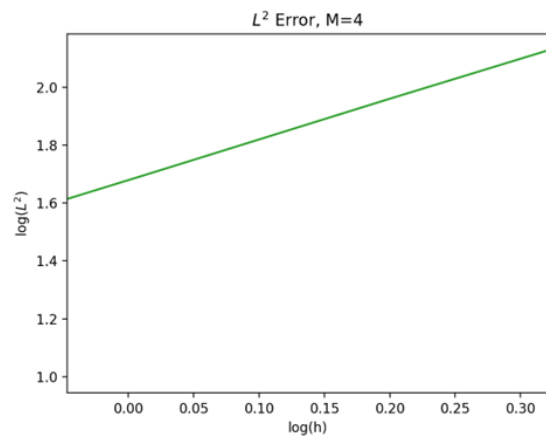
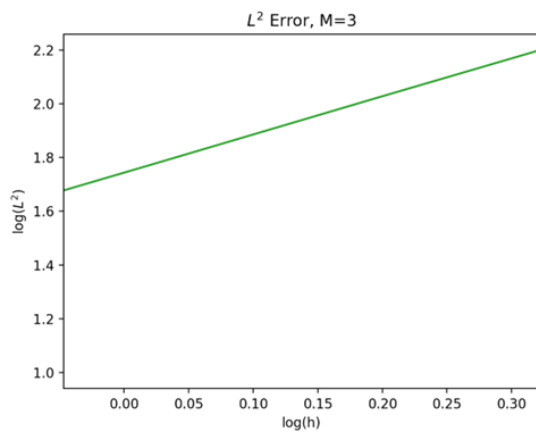






As expected, as the number of elements increase, both basis functions provide a smoother approximation that matches the analytical solution better.

The L^2 error plotted as a function of elements are included below. Of note, the quadratic approximation ($M = 4$) has a lower y-intercept as $\log(h)$ approaches 0 than the cubic approximation ($M=3$).



Works Cited

Whiteley, J. (2017). *Finite element methods: A Practical Guide*. Springer.

Appendix

```
import numpy as np
import pandas as pd
import math
from sympy import diff, integrate, simplify
import sympy as sp
import matplotlib.pyplot as plt
from scipy.integrate import simpson

#Number of elements
N = 10
M_vector = [3,4]
def u_analytical(x):
    term1 = np.exp(np.pi)*np.sin(x/2)
    term2 = (2*(3-np.exp(2*np.pi))/np.exp(np.pi))*np.cos(x/2)
    return 2*np.exp(-x)*(term1+term2)+np.sin(x)

def L_e(U,u,x,N,nodes):
    aux = 0
    for i in range(N):
        y_values = U[2*i:2*i+nodes]-u[2*i:2*i+nodes]
        x_values = x[2*i:2*i+nodes]
        aux+= (simpson(y_values, x_values))**(2)

    return (aux)**(0.5)

plt.plot(1)
for M in M_vector:
    #Number of nodes
    n=((M+1)*N)+1

    xi = sp.Symbol('xi')

    # Define the mapping from x in [0,1] to xi in [-1,1]
    x = (xi + 1)/2
```

```

# Nodes in the physical domain
x_var = [float(val) for val in np.linspace(0, 1, M+2)]
x_var = [sp.symbols(val) for val in x_var]
phi = [sp.S.Zero for _ in range(M+2)]
for j in range(2+M):
    aux=1
    for m in range(M+2):
        if m !=j:
            aux= aux*(x-x_var[m])/(x_var[j]-x_var[m])
    phi[j]=sp.simplify(aux)

#length or hight of each element
h=(2*math.pi)/(N)

#Creating local stiffness matrix
K_local = np.zeros((M+2,M+2),dtype=float)

#Creating local length
h_local = (2*math.pi)/(n-1)

#Creating the global force vector
f = np.zeros(n)

#Creating the global stiffness matrix
K = np.zeros((n,n),dtype=float)

for e in range(N):
    #Defining local stiffness matrix at each element
    x_min = -math.pi+h*e
    x_max = -math.pi+h*(e+1)
    je = (x_max-x_min)/2
    x = je*xi+(x_min+x_max)/2
    for i in range(0,(M+2)):
        for j in range(0,(M+2)):
            aux = (-4*(1/je**2)*diff(phi[i],xi)*diff(phi[j],xi)+8*(1/je)*phi[i]*diff(phi[j],xi)+5*phi[i]*p
hi[j])*je
            K_local[i,j] = float(integrate(simplify(aux), (xi, -1,1)))

#Assembling local force vector
f_local = np.zeros(((M+2)))
for i in range(len(f_local)):
    f_local[i]=float(integrate((sp.sin(x)+8*sp.cos(x))*phi[i]*je,(xi, -1,1)).evalf())

```

```

#Assembling the global stiffness matrix
global_dofs = np.zeros(((M+2)))
for i in range(M+2):
    global_dofs[i] =int((M+1)*e+i)

for i in range(M+2):
    for j in range(len(K_local[0,:])):
        K[int(global_dofs[i]), int(global_dofs[j])] += K_local[i, j]
f[int(global_dofs[0]):int(global_dofs[-1]+1)]+=f_local

##For boundary conditions:
f[0]+=-20
u_pi=2
for j in range(0,n-1):
    f[j]-= u_pi*K[j,-1]

u=np.linalg.solve(K[0:-1,0:-1],f[0:-1])
x_analytical =np.linspace(-np.pi,np.pi,len(u))
u_an=u_analytical(x_analytical)
nodes = M+2
plt.plot(x_analytical,u,label=f'M={M}')

plt.title(f'Quadratic Basis Function ({N} Elements)')
plt.plot(x_analytical,u_an,label='Analytical')
plt.xlabel('Position, x')
plt.ylabel('u')
plt.xlim([-np.pi,np.pi])
plt.legend()
plt.show()

```